Laboratory 4
Convolutional Neural Networks

1. Convolutional Neural Networks

- a **convolutional neural network**, also known as CNN or ConvNet, is a class of neural networks designed specifically for processing image data;
- a CNN typically includes three layers:
  1. *convolutional layer*;
  2. *pooling layer*;
  3. *fully-connected layer*.
- the convolutional layer is the first layer of a convolutional network. While convolutional layers can be followed by additional convolutional layers or pooling layers, the fully-connected layer is the final layer.

1.1 Convolutions for images

- the **convolutional layer** is the core building block of a CNN, and it is where the majority of computation occurs;
- in a convolutional layer, an input tensor and a kernel tensor are combined to produce an output tensor through a **cross-correlation operation**;

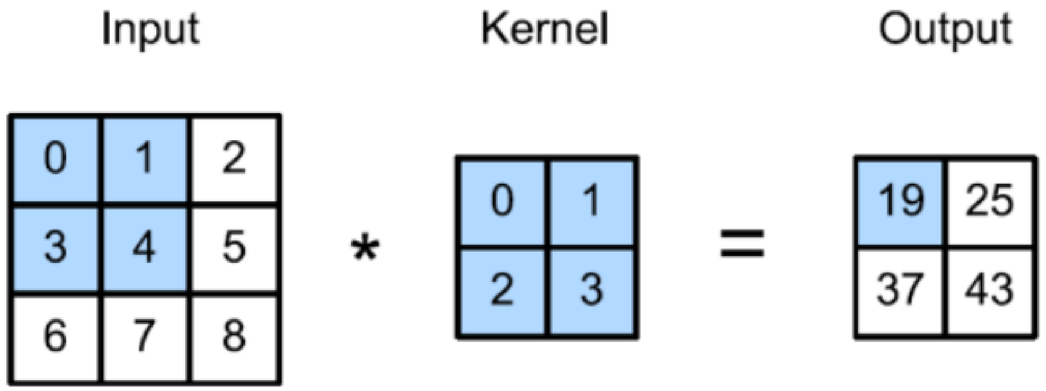


Figure 1: Cross-correlation operation. The first output element is computed as $0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19$.

1.1 Convolutions for images

- Figure 1 has an input tensor of shape 3×3 and a kernel/filter of shape 2×2;
- in the cross-correlation operation, we begin with the kernel positioned at the upper-left corner of the input tensor, and slide it across the input tensor, both from left to right and top to bottom;
- when the kernel is applied to an area of the input, a dot product is calculated between the input elements and the kernel;
- the final output from the series of dot products from the input and the kernel is known as a **feature map**, **activation map**, or a **convolved feature**;
- the *output size* is given by the input size $n_h \times n_w$ minus the size of the convolution kernel $k_h \times k_w$ via:

$$(n_h - k_h + 1) \times (n_w - k_w + 1).$$

2. Padding and stride

- **stride** is the distance that the kernel moves over the input matrix;

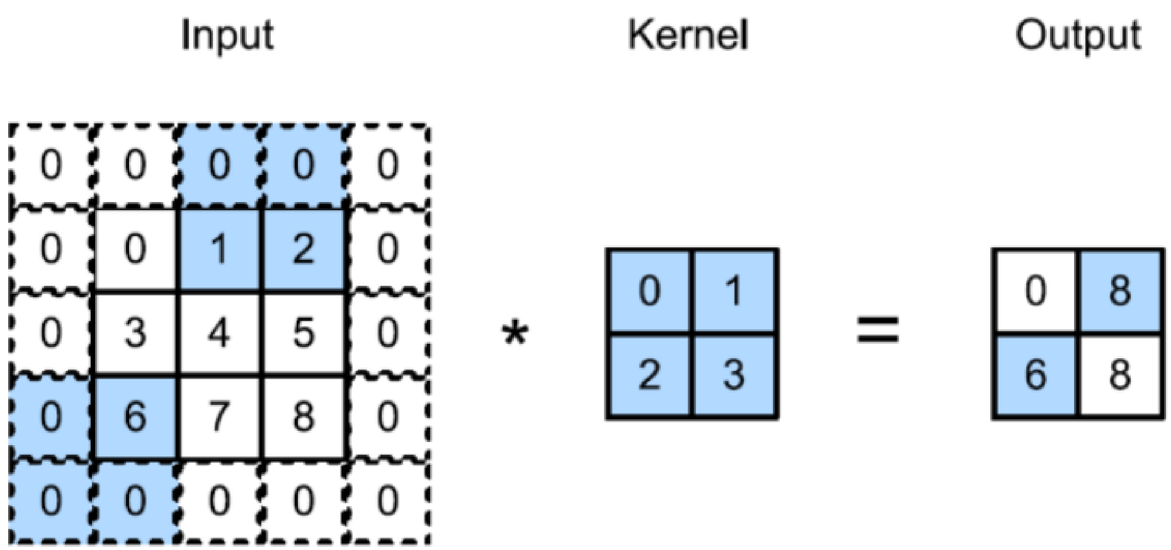


Figure 3: Cross-correlation operation with strides of 3 and 2 for height and width, respectively.

2.1 Padding and stride in PyTorch

- padding:

```
X = torch.rand(1, 1, 8, 8)
# Note that here 1 row or column is padded on either side, so a total of 2
# rows or columns are added
conv2d = nn.Conv2d(1, 1, kernel_size=3, padding=1)
#conv2d = nn.Conv2d(1, 1, kernel_size=(3,3), padding=(1,1))
conv2d(X).shape
```

```
torch.Size([1, 1, 8, 8])
```

- stride:

```
X = torch.rand(1, 1, 8, 8)
conv2d = nn.Conv2d(1, 1, kernel_size=3, padding=1, stride=2)
conv2d(X).shape
```

```
torch.Size([1, 1, 4, 4])
```

3. Multiple input and multiple output channels

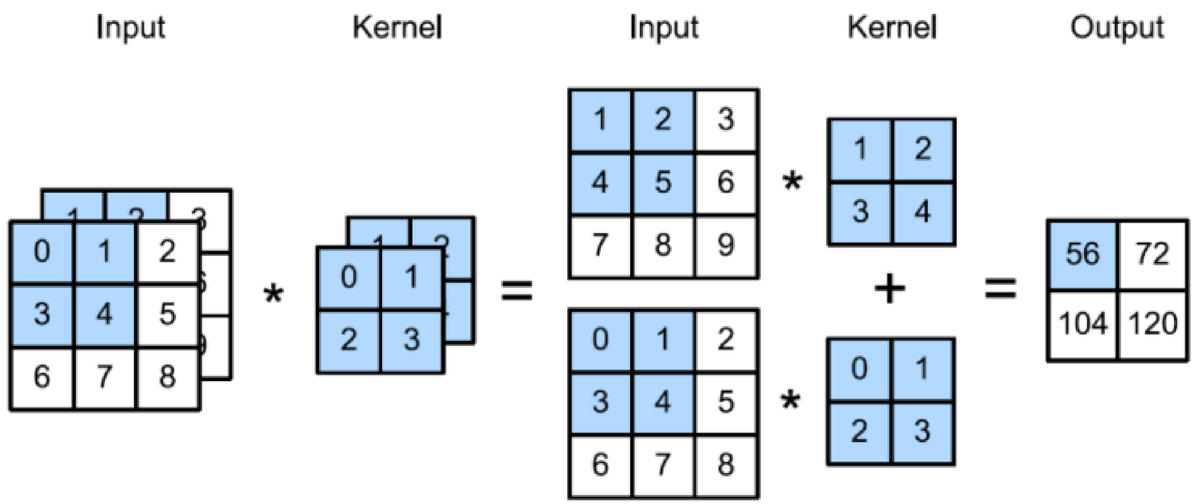


Figure 4: Cross-correlation operation with 2 input channels, $c_{in} = 2$ in PyTorch.

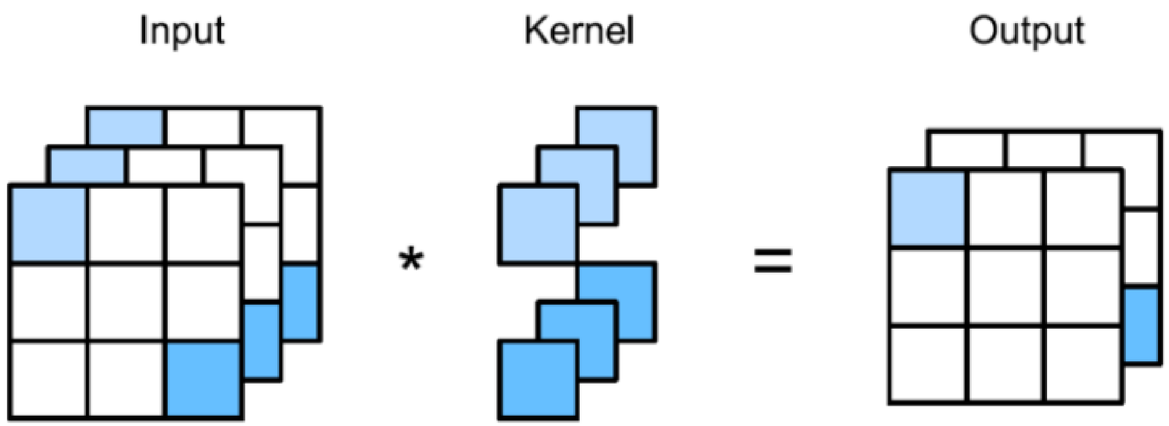


Figure 5: The cross-correlation operation uses 1×1 kernel with 3 input channels and 2 output channels ($c_{in} = 3$ and $c_{out} = 2$).

4. Pooling

- **pooling layer** makes the representations smaller and more manageable;
- the pooling layer contains no parameters (there is no kernel);

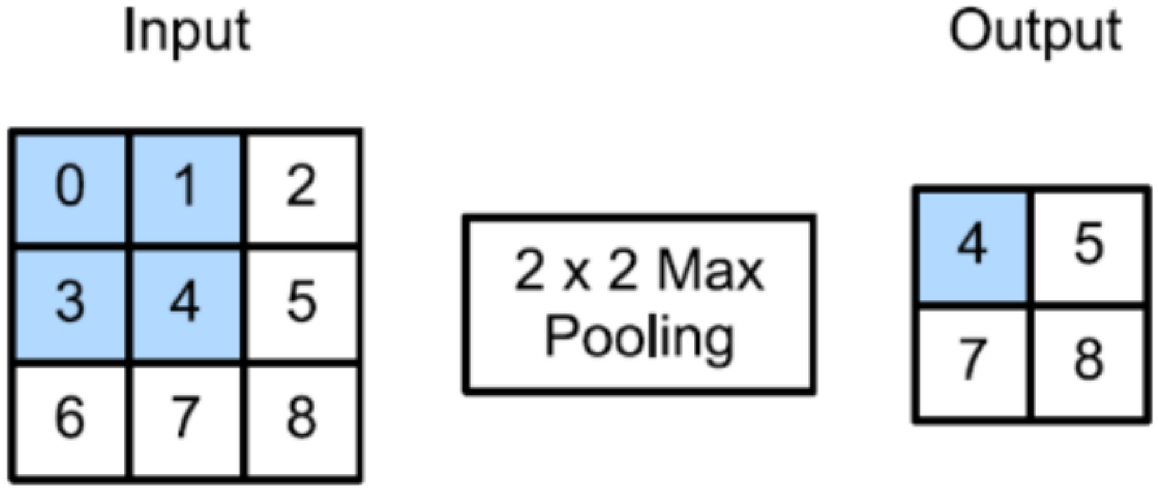


Figure 6: Maximum pooling with a pooling window shape of 2×2. The first output element is computed as $max(0, 1, 3, 4) = 4$.

4.1 Pooling layer in PyTorch

```
X = torch.arange(16, dtype=torch.float32).reshape((1, 1, 4, 4))
print(X)

pool2d = nn.MaxPool2d(3)
pool2d(X)
```

```
tensor([[[[ 0.,  1.,  2.,  3.],
          [ 4.,  5.,  6.,  7.],
          [ 8.,  9., 10., 11.],
          [12., 13., 14., 15.]]]])
tensor([[[[10.]]]])
```

- the stride and the pooling window in the *nn.MaxPool2d* class have the same shape. Above, we use a pooling window of shape 3×3, so we get a stride shape of 3×3, by default.

5. LeNet architecture

- LeNet – the first convolutional network;

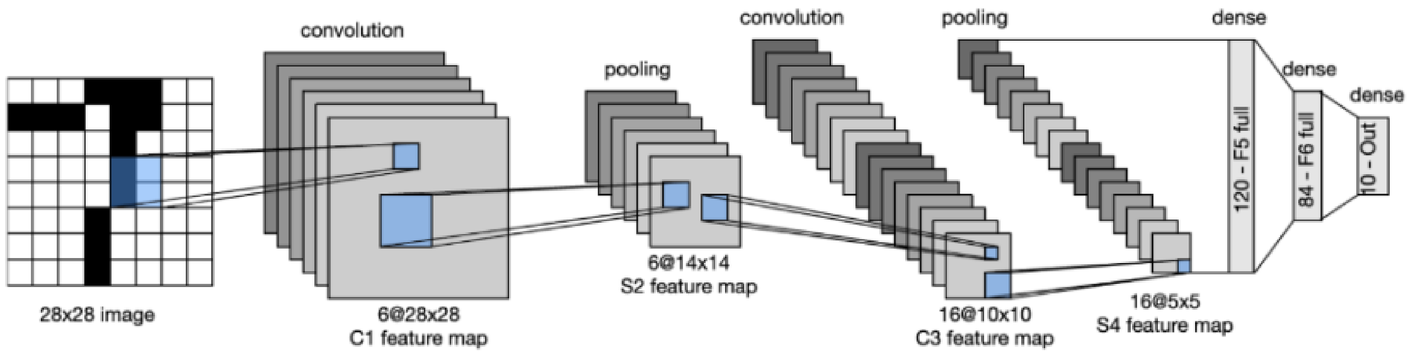


Figure 7: LeNet architecture. The input is a handwritten digit and the output is a probability over 10 possible outcomes.

- LeNet implementation in PyTorch:

```
net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10))
```

5. LeNet architecture

- we can pass a single-channel 28×28 image through the network and print the output shape at each layer;

```
X = torch.rand(1, 1, 28, 28)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
Conv2d output shape:     torch.Size([1, 6, 28, 28])
Sigmoid output shape:    torch.Size([1, 6, 28, 28])
AvgPool2d output shape:  torch.Size([1, 6, 14, 14])
Conv2d output shape:     torch.Size([1, 16, 10, 10])
Sigmoid output shape:    torch.Size([1, 16, 10, 10])
AvgPool2d output shape:  torch.Size([1, 16, 5, 5])
Flatten output shape:    torch.Size([1, 400])
Linear output shape:     torch.Size([1, 120])
Sigmoid output shape:    torch.Size([1, 120])
Linear output shape:     torch.Size([1, 84])
Sigmoid output shape:    torch.Size([1, 84])
Linear output shape:     torch.Size([1, 10])
```

6. AlexNet

- A. Krizhevsky, I. Sutskever, G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks", *NIPS*, 2012.
- AlexNet contains eight layers, the first five are convolutional and the remaining three are fully-connected;

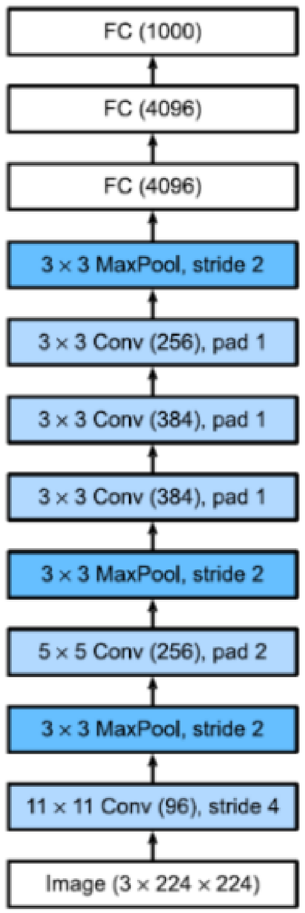


Figure 8: AlexNet architecture.

6. AlexNet

```
net = nn.Sequential(
    # Here, we use a larger 11 x 11 window to capture objects. At the same
    # time, we use a stride of 4 to greatly reduce the height and width of the
    # output. Here, the number of output channels is much larger than that in
    # LeNet
    nn.Conv2d(1, 96, kernel_size=11, stride=4, padding=1), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    # Make the convolution window smaller, set padding to 2 for consistent
    # height and width across the input and output, and increase the number of
    # output channels
    nn.Conv2d(96, 256, kernel_size=5, padding=2), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    # Use three successive convolutional layers and a smaller convolution
    # window. Except for the final convolutional layer, the number of output
    # channels is further increased. Pooling layers are not used to reduce the
    # height and width of input after the first two convolutional layers
    nn.Conv2d(256, 384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 256, kernel_size=3, padding=1), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Flatten(),
    # Here, the number of outputs of the fully-connected layer is several
    # times larger than that in LeNet. Use the dropout layer to mitigate
    # overfitting
    nn.Linear(6400, 4096), nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(4096, 4096), nn.ReLU(),
    nn.Dropout(p=0.5),
    # Output layer. Since we are using Fashion-MNIST, the number of classes is
    # 10, instead of 1000 as in the paper
    nn.Linear(4096, 10))
```

7. Networks using blocks (VGG)

- the basic building block of classic CNNs is a sequence of the following:
  - a convolutional layer with padding to maintain the resolution;
  - a nonlinearity such as ReLU;
  - a pooling layer such as a maximum pooling layer.
- one VGG block consists of a sequence of convolutional layers, followed by a maximum pooling layer for spatial downsampling;

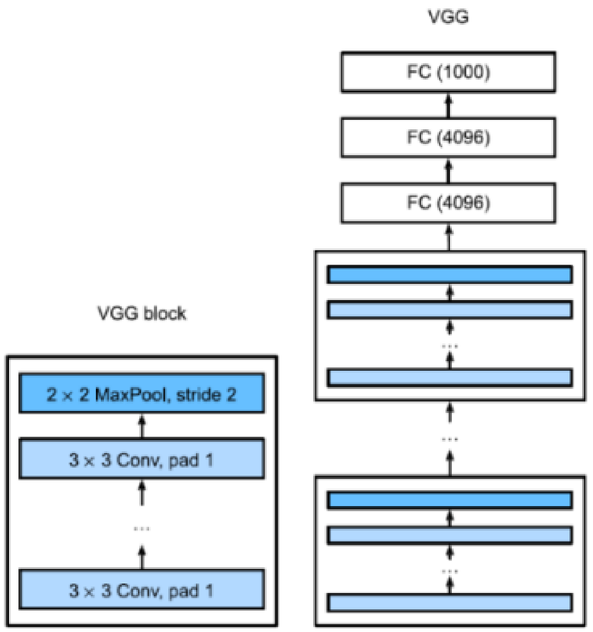


Figure 9: VGG architecture.

7. Networks using blocks (VGG)

- VGG block:

```
def vgg_block(num_convs, in_channels, out_channels):
    layers = []
    for _ in range(num_convs):
        layers.append(nn.Conv2d(in_channels, out_channels,
                                kernel_size=3, padding=1))
        layers.append(nn.ReLU())
        in_channels = out_channels
    layers.append(nn.MaxPool2d(kernel_size=2,stride=2))
    return nn.Sequential(*layers)
```

- VGG architecture:

```
conv_arch = ((1, 64), (1, 128), (2, 256), (2, 512), (2, 512))

def vgg(conv_arch):
    conv_blks = []
    in_channels = 1
    # The convolutional part
    for (num_convs, out_channels) in conv_arch:
        conv_blks.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels

    return nn.Sequential(
        *conv_blks, nn.Flatten(),
        # The fully-connected part
        nn.Linear(out_channels * 7 * 7, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 10))

net = vgg(conv_arch)
net
```

7. Networks using blocks (VGG)

```
Sequential(
  (0): Sequential(
    (0): Conv2d(1, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (1): Sequential(
    (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (2): Sequential(
    (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU()
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (3): Sequential(
    (0): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU()
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (4): Sequential(
    (0): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU()
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (5): Flatten(start_dim=1, end_dim=-1)
  (6): Linear(in_features=25088, out_features=4096, bias=True)
  (7): ReLU()
  (8): Dropout(p=0.5, inplace=False)
  (9): Linear(in_features=4096, out_features=4096, bias=True)
  (10): ReLU()
  (11): Dropout(p=0.5, inplace=False)
  (12): Linear(in_features=4096, out_features=10, bias=True)
```

8. Batch normalization

- training deep neural networks is difficult, and getting them to converge in a reasonable amount of time can be challenging;
- thus, techniques that accelerates the convergence of deep networks are proposed such as **batch normalization**;
- batch normalization is applied to individual layers and works as follows: in each training iteration, we first normalize the inputs by subtracting their mean and dividing by their standard deviation, where both are estimated based on the statistics of the current mini-batch;
- next, we apply a scale coefficient and a scale offset;

8. Batch normalization

- denoting by $\mathbf{x} \in \mathcal{B}$ an input to batch normalization (BN) that is from a mini-batch $\mathcal{B}$, batch normalization transforms $\mathbf{x}$ according to the following expression:

$$BN(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} + \beta.$$

- $\hat{\mu}_{\mathcal{B}}$ is the sample mean ($\hat{\mu}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|}\sum_{\mathbf{x}\in\mathcal{B}}\mathbf{x}$) and $\hat{\sigma}_{\mathcal{B}}$ is the sample standard deviation ($\hat{\sigma}^2_{\mathcal{B}} = \frac{1}{|\mathcal{B}|}\sum_{\mathbf{x}\in\mathcal{B}}(\mathbf{x} - \hat{\mu}_{\mathcal{B}})^2 + \epsilon$, $\epsilon > 0$) of the mini-batch $\mathcal{B}$;
- γ and β are learnable parameters;

8. Batch normalization

- batch normalization applied to fully-connected layers:

$$\mathbf{h} = \phi(BN(\mathbf{W}\mathbf{x} + \mathbf{b})),$$

where $\mathbf{x}$ is the input to the fully-connected layer, $\mathbf{W}\mathbf{x} + \mathbf{b}$ denotes the affine transformation and ϕ is the activation function;

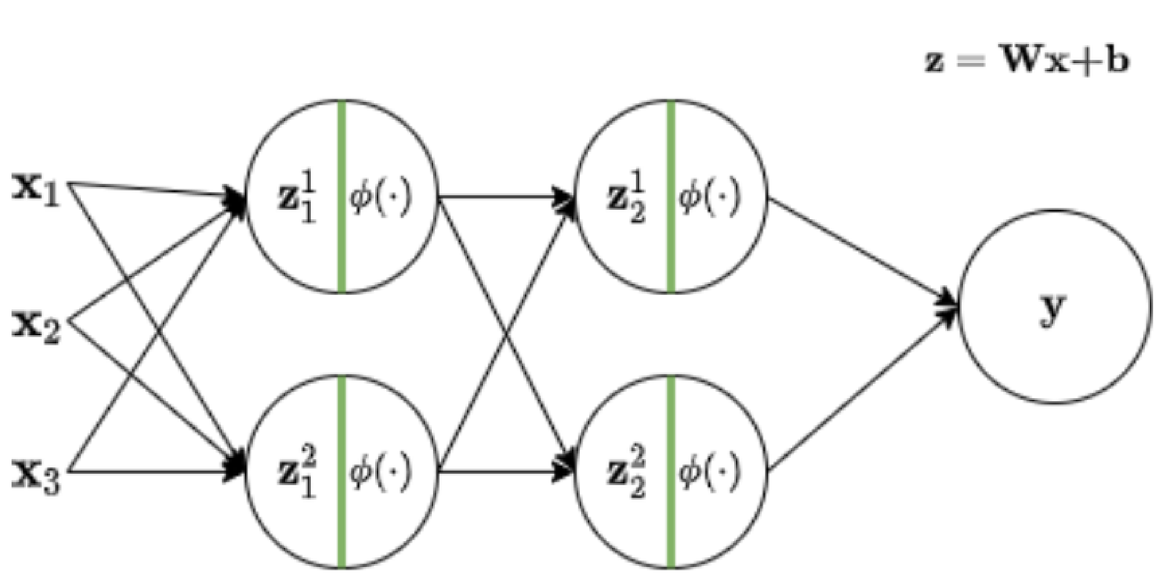


Figure 10: Batch Normalization – represented with a green line – is applied to the neurons' output before applying the activation function.

8. Batch normalization

- similarly, with convolutional layers, we can apply batch normalization after the convolution and before the nonlinear activation function;
- when the convolution has multiple output channels, we need to carry out batch normalization for each of the outputs of these channels, and each channel has its own scale and shift parameters, both of which are scalars;

8.1 Batch normalization in PyTorch

- the corresponding layers in PyTorch are:
  - *nn.BatchNorm2d* for batch normalization applied after convolutional layers;
  - *nn.BatchNorm1d* for batch normalization applied after fully-connected layers;

The first parameter represents the number of output channels of the previous layer.

```
net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5), nn.BatchNorm2d(6), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.BatchNorm2d(16), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2), nn.Flatten(),
    nn.Linear(256, 120), nn.BatchNorm1d(120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.BatchNorm1d(84), nn.Sigmoid(),
    nn.Linear(84, 10))
```

9. Residual networks (ResNet)

- the solid line carrying the layer input **x** to the addition operator is called a *residual connection* (or *shortcut connection*);

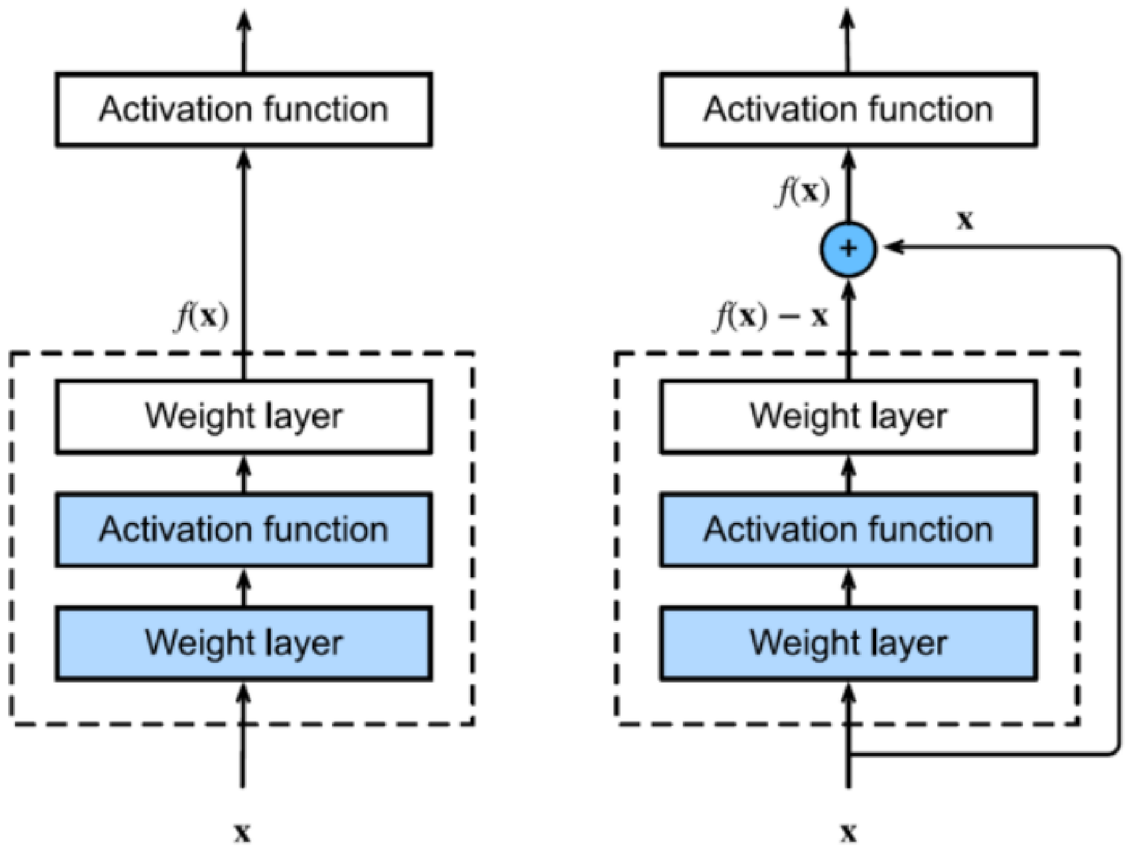


Figure 11: A regular block (left) and a residual block (right).

9.1 ResNet in PyTorch

```
class Residual(nn.Module):
    """The Residual block of ResNet."""
    def __init__(self, input_channels, num_channels,
                 use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.Conv2d(input_channels, num_channels,
                               kernel_size=3, padding=1, stride=strides)
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.conv2 = nn.Conv2d(num_channels, num_channels,
                               kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(num_channels)
        if use_1x1conv:
            self.conv3 = nn.Conv2d(input_channels, num_channels,
                                   kernel_size=1, stride=strides)
        else:
            self.conv3 = None

    def forward(self, X):
        Y = nn.ReLU()(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3:
            X = self.conv3(X)
        Y += X
        return nn.ReLU()(Y)
```

10. Exercise

Calculate the number of learnable parameters for the convolutional neural network given below. The input image has a shape of $1 \times 32 \times 32$.

```
net = nn.Sequential(
    nn.Conv2d(1, 3, kernel_size=5, padding=2),
    nn.BatchNorm2d(3),
    nn.MaxPool2d(kernel_size=6),
    nn.Flatten(),
    nn.Linear(75, 5))
```